Problem Statement: Finding the Longest Subarray with Equal Numbers of 0s and 1s

- Description: Given a binary array (containing only 0s and 1s), write a C# function to find the length of the longest contiguous subarray with an equal number of 0s and 1s.
- Example:
  - Input: [0, 1, 0, 1, 0, 1, 1]
  - Output: 6 (The longest subarray is [0, 1, 0, 1, 0, 1])

1. Solution without using Hashmap/Dictionary

1. **Transform the Array**:
   - Convert all `0`s in the array to `-1`s. This change allows us to use the cumulative sum technique to find subarrays with an equal number of `0`s and `1`s by looking for subarrays where the cumulative sum is zero.
2. **Cumulative Sum**:
   - As we iterate through the array, we maintain a cumulative sum. The idea is that if we encounter the same cumulative sum at two different indices, the elements between these two indices form a subarray with an equal number of `0`s and `1`s. This is because the net change in cumulative sum between these two indices is zero, indicating an equal number of `-1`s (originally 0s) and `1`s.
3. **Use a HashMap**:
   - Use a hashmap (dictionary) to store the first occurrence of each cumulative sum. The key will be the cumulative sum, and the value will be the index at which this sum first appears.
4. **Calculate Maximum Length**:
   - While iterating through the array, calculate the cumulative sum. If the cumulative sum has been seen before, the subarray between the previous occurrence and the current index has a sum of zero (equal number of `0`s and

1s). Update the maximum length of such subarrays accordingly.

Step-by-Step Example:

Consider the input array [0, 1, 0, 1, 0, 1, 1].

1. **Transform the Array**:
   - Convert 0s to -1s: [-1, 1, -1, 1, -1, 1, 1].
2. **Initialize Variables**:
   - cumulativeSum = 0
   - maxLength = 0
   - sumIndexMap = {0: -1} (to handle subarrays starting from index 0)
3. **Iterate Through the Array**:
   - Index 0: cumulativeSum = -1, update sumIndexMap = {0: -1, -1: 0}
   - Index 1: cumulativeSum = 0, update maxLength = 2 (subarray [0, 1])
   - Index 2: cumulativeSum = -1, seen before at index 0, update maxLength = 2
   - Index 3: cumulativeSum = 0, update maxLength = 4 (subarray [0, 1, 0, 1])
   - Index 4: cumulativeSum = -1, seen before at index 0, update maxLength = 4
   - Index 5: cumulativeSum = 0, update maxLength = 6 (subarray [0, 1, 0, 1, 0, 1])
   - Index 6: cumulativeSum = 1, update sumIndexMap = {0: -1, -1: 0, 1: 6}

```
using System;
using System.Collections.Generic;

public class Solution
{
    public int FindMaxLength(int[] nums)
    {
        int n = nums.Length;
```

```
        // Convert 0s to -1s
        for (int i = 0; i < n; i++)
        {
            if (nums[i] == 0)
            {
                nums[i] = -1;
            }
        }

        // Dictionary to store the first occurrence of each cumulative sum
        Dictionary<int, int> sumIndexMap = new Dictionary<int, int>();
        sumIndexMap[0] = -1; // Initialize to handle case when subarray starts from
index 0

        int maxLength = 0;
        int cumulativeSum = 0;

        for (int i = 0; i < n; i++)
        {
            cumulativeSum += nums[i];

            if (sumIndexMap.ContainsKey(cumulativeSum))
            {
                // If this cumulative sum has been seen before, calculate the length of
subarray
                int prevIndex = sumIndexMap[cumulativeSum];
                maxLength = Math.Max(maxLength, i - prevIndex);
            }
            else
            {
                // Store the first occurrence of this cumulative sum
                sumIndexMap[cumulativeSum] = i;
            }
        }

        return maxLength;
    }

    public static void Main(string[] args)
    {
        Solution solution = new Solution();
        int[] nums = { 0, 1, 0, 1, 0, 1, 1 };
        int result = solution.FindMaxLength(nums);
        Console.WriteLine("The length of the longest contiguous subarray with
equal numbers of 0s and 1s is: " + result);
```

```
    }
}
```

Explanation of the Code:

- **Transform the Array**: Convert `0`s to `-1`s.
- **Cumulative Sum and HashMap**: Use a dictionary to store the first occurrence of each cumulative sum.
- **Iterate and Calculate Maximum Length**: Update the cumulative sum while iterating and calculate the maximum length of subarrays with a sum of zero, using the hashmap to find previous occurrences of the same cumulative sum.
- **Result**: Return the maximum length found.

This approach ensures that the solution runs in O(n) time complexity and uses O(n) space complexity.

2. Solution using Dictionary

Solution Explanation:

1. **Transform the Array**:
    - Convert all `0`s in the array to `-1`. This way, the problem of finding equal numbers of `0`s and `1`s is transformed into finding a subarray whose sum is zero.
2. **Use a HashMap**:
    - Use a hashmap to store the first occurrence of each cumulative sum. The key will be the cumulative sum, and the value will be the index at which this sum first appears.
3. **Iterate through the Array**:
    - Calculate the cumulative sum as you iterate through the array.
    - If the cumulative sum is zero, it means that all the elements from the start of the array to the current index form a subarray with equal numbers of `0`s and `1`s.

- If the cumulative sum has been seen before, it means there is a subarray between the previous occurrence of this sum and the current index which sums to zero (i.e., it has equal numbers of `0`s and `1`s).
- Track the maximum length of such subarrays.

```
using System;
using System.Collections.Generic;

public class Solution
{
    public int FindMaxLength(int[] nums)
    {
        // Dictionary to store the first occurrence of each cumulative sum
        Dictionary<int, int> sumIndexMap = new Dictionary<int, int>();
        sumIndexMap[0] = -1; // Initialize to handle case when subarray starts from index 0

        int maxLength = 0;
        int cumulativeSum = 0;

        for (int i = 0; i < nums.Length; i++)
        {
            // Convert 0 to -1 to transform the problem
            cumulativeSum += (nums[i] == 0) ? -1 : 1;

            if (sumIndexMap.ContainsKey(cumulativeSum))
            {
                // If this cumulative sum has been seen before, calculate the length of subarray
                int prevIndex = sumIndexMap[cumulativeSum];
                maxLength = Math.Max(maxLength, i - prevIndex);
            }
            else
            {
                // Store the first occurrence of this cumulative sum
                sumIndexMap[cumulativeSum] = i;
            }
        }

        return maxLength;
    }
```

```
    public static void Main(string[] args)
    {
        Solution solution = new Solution();
        int[] nums = { 0, 1, 0, 1, 0, 1, 1 };
        int result = solution.FindMaxLength(nums);
        Console.WriteLine("The length of the longest contiguous subarray with
equal numbers of 0s and 1s is: " + result);
    }
}
```

Explanation of the Code:

- **Initialization**: A dictionary `sumIndexMap` is initialized to store cumulative sums and their first occurrence indices. The sum `0` is mapped to `-1` to handle the case where the subarray starts from the beginning of the array.
- **Iteration**: The array is iterated through, and the cumulative sum is updated by adding `1` for `1` and subtracting `1` for `0`.
- **Checking Cumulative Sum**: If the cumulative sum has been seen before, the length of the subarray from the previous occurrence of this sum to the current index is calculated and compared to the maximum length found so far.
- **Updating Dictionary**: If the cumulative sum has not been seen before, it is added to the dictionary with the current index.
- **Result**: After iterating through the array, the maximum length of the subarray with equal numbers of `0`s and `1`s is returned.

This solution has a time complexity of O(n) and a space complexity of O(n), where n is the length of the input array.